

CONFIDENTIAL DOCUMENT

Tawveeri PDR Implementation Status Report

Report Date: 2026-02-18

Document Type	Development Status Report
Project	Tawveeri — Price Comparison Platform
Prepared By	Etlaq — etlaq.sa
Version	1.0
Date	18 February 2026

About Etlaq

Etlaq is a Saudi-based software development company specializing in building modern digital platforms, web applications, and mobile solutions for businesses across the Kingdom. We transform ideas into scalable, production-grade products.

etlaq.sa

Tawveeri PDR Implementation Status Report

Report Date: 2026-02-18 Branch: `phase2_v1` Compared Against: 10-Week Implementation Plan (PDR.pdf)

Executive Summary

CATEGORY	IMPLEMENTED	PARTIAL	NOT IMPLEMENTED	TOTAL	COMPLETION
Functional Requirements	29	5	7	41	77%
Non-Functional Requirements	5	6	5	16	50%
Overall	34	11	12	57	69%

Partial items are counted as 50% in the overall percentage.

Week-by-Week Breakdown

Week 1 — Foundation & Setup (95%)

TASK	STATUS	NOTES
Development environment setup	✓ Done	Next.js 15 + Turbopack, Tailwind v4, TypeScript
Database schema design (Users, Products, Stores, Transactions)	✓ Done	Full schema in <code>scripts/database/01-schema.sql</code> — users, products, stores, product_stores, price_history, transactions, notifications, admin_logs, etc.
Server & API setup	✓ Done	Next.js App Router API routes (<code>src/app/api/</code>)
Security baseline (SSL, Encryption)	✓ Done	Supabase handles SSL; secure auth via <code>@supabase/ssr</code>
Daily backup system + Monitoring & Logging	● Partial	Audit logging fully implemented (<code>src/lib/auth/audit.ts</code>). No evidence of automated daily backups (relies on Supabase managed backups).
RLS (Row-Level Security) setup	✓ Done	<code>scripts/database/02-rls-policies.sql</code> applied
User roles planning (admin, customer, store)	✓ Done	4 roles: <code>admin</code> , <code>customer</code> , <code>store</code> , <code>guest</code> in <code>src/lib/database/types.ts</code>

Week 2 — User Accounts & Access (100%)

TASK	STATUS	NOTES
User registration (Email/Phone/Google)	✓ Done	Email + password, Phone OTP, Google/Facebook/Apple OAuth in <code>src/lib/auth/auth-context.tsx</code>
Guest access	✓ Done	Search/browse works without account; public routes unprotected
Account management (profile editing, password reset, delete account)	✓ Done	Profile page with avatar upload, password change, account deletion, email/phone verification (<code>src/app/[locale]/(dashboard)/profile/page.tsx</code>)
Legal compliance (Saudi data privacy laws)	✓ Done	Privacy policy + Terms of Service pages exist bilingual (<code>src/app/[locale]/(public)/privacy/</code> , <code>src/app/[locale]/(public)/terms/</code>)
User/Admin/Customer page creation (frontend access layer)	✓ Done	Route groups: <code>(public)</code> , <code>(dashboard)</code> , <code>/admin/</code> , <code>/store/</code> ; middleware enforces roles
Phone password reset	✓ Done	3-step flow in <code>forgot-password/page.tsx</code> : phone input → OTP verification (6-digit with auto-advance) → new password form. Backend: <code>POST /api/auth/reset-password-phone</code> verifies OTP from <code>phone_otps</code> table + updates password via <code>supabase.auth.admin.updateUserById()</code> . Includes notification + audit log.

Week 3 — Search & Filtering (95%)

TASK	STATUS	NOTES
Product search by category (TV, Laptop, Smartphone)	✓ Done	Category passed end-to-end: SearchPage → <code>POST /api/search/scrape</code> body → <code>searchAllStores(query, stores, pages, sort, category)</code> → filtered by <code>matchesCategory()</code> from <code>src/lib/scraping/utils/category-utils.ts</code>
Filtering by brand, model	✓ Done	Dynamic brand filter from DB in <code>src/components/search/filter-sidebar.tsx</code>
Filtering by RAM, storage, size, resolution, color	✓ Done	Dynamic spec filters per category in <code>filter-sidebar.tsx</code> using <code>CATEGORY_SPEC_FILTERS</code> from <code>src/lib/scraping/config/spec-configs.ts</code> . Specs extracted client-side from product titles via <code>extractSpecsFromTitle()</code> . Covers RAM, storage, screen size, resolution, panel type, audio type, wireless/ANC across 6 categories (smartphone, laptop, tv, tablet, audio, gaming). Discount %, condition, and shipping speed filters also wired.
Dynamic filters (based on category)	✓ Done	Spec filter sections change dynamically based on selected category. Smartphone shows RAM/storage, laptop shows CPU/GPU/storage type, TV shows resolution/panel type, audio shows type/wireless/ANC, etc.
Sort by price, popularity, rating, store	✓ Done	4 sort options + 7 filter types with URL state persistence
Search suggestions (auto-suggest)	✓ Done	Debounced autocomplete from DB products (<code>src/components/search/search-bar.tsx</code>)
Search history log	✓ Done	Auto-logged to <code>search_history</code> table; visible in search bar + saved searches

Week 4 — Store Integration (95%)

TASK	STATUS	NOTES
Integration with 5 major stores (Extra, Jarir, Almanea, Noon, Amazon.sa)	✓ Done	All 5 stores fully operational. Search scrapers work for all 4 main stores. Cron scrapers implemented for all 5: <code>JarirScraper</code> (existing), <code>NoonScraper</code> (JSON API at <code>/_svc/catalog/api/v3/</code>), <code>AmazonScraper</code> (cheerio + Puppeteer, ASIN-based), <code>ExtraScraper</code> (<code>__NEXT_DATA__</code> JSON + HTML fallback), <code>AlmaneaScraper</code> (HTML selectors + JSON-LD). Orchestrator switch in <code>scraping-orchestrator.ts</code> routes all stores.
Localized store integration (additional Saudi stores)	✓ Done	5 Saudi stores with bilingual configs (<code>name_ar</code> , <code>name_en</code>) in <code>src/lib/scraping/config/store-configs/</code> . All with working cron + search scrapers.
Store onboarding portal (self-service for stores)	✓ Done	Full store portal: <code>/store/dashboard</code> , <code>/store/products</code> , <code>/store/analytics</code> , <code>/store/transactions</code> with product management
Store rating system	✓ Done	<code>store_reviews</code> table with multi-field ratings (delivery, quality, service); review form + display on store pages

Week 5 — Product Experience (95%)

TASK	STATUS	NOTES
Product detail page (images, specs, prices)	✓ Done	Full page with image gallery, carousel, full-screen modal, specs, multi-store pricing (<code>src/app/[locale]/(public)/products/[slug]/page.tsx</code>)
Product availability checker	✓ Done	4 statuses: <code>in_stock</code> , <code>out_of_stock</code> , <code>limited_stock</code> , <code>pre_order</code> with colored badges
Product videos/demos	✓ Done	<code>ProductVideoPlayer</code> component supports YouTube embeds + direct video files
Price comparison engine	✓ Done	Per-store pricing cards with best-price highlighting, savings badges, affiliate links
Price history tracking	✓ Done	<code>price_history</code> table + SVG chart with 30/90/365-day ranges and trend indicators
Multi-language support (Arabic/English)	✓ Done	19 translation namespace files, <code>SimpleIntlProvider</code> , RTL/LTR auto-switching

TASK	STATUS	NOTES
Currency support (SAR)	✅ Done	<code><Price></code> component with SAR SVG symbol; <code>formatPrice()</code> utilities

Week 6 — Personalization & Favorites (55%)

TASK	STATUS	NOTES
Wishlist / Save products	✅ Done	Full CRUD with notes in <code>src/app/[locale]/(dashboard)/wishlist/page.tsx</code>
Favorites sync (cross-device)	✅ Done	Supabase-backed; syncs automatically when user logs in on another device
Search history log	✅ Done	(Covered in Week 3)
Personalized recommendations	🟡 Partial	Category-based + <code>view_count</code> algorithm on dashboard; not AI/ML-powered
AI-powered smart recommendations	❌ Not Done	No ML models, collaborative filtering, or AI recommendation engine

Week 7 — Notifications & Engagement (55%)

TASK	STATUS	NOTES
Notification system (price drop, back in stock)	✅ Done	Full in-app notification system with bilingual content; cron job for price alert checks
Email/SMS alerts	🟡 Partial	Email templates built but backend uses placeholder <code>supabase.functions.invoke('send-email')</code> — no actual email service connected . SMS only for OTP, not alerts.
Push notifications (mobile)	🟡 Partial	UI preferences toggles exist in settings; no service worker or Web Push API backend
Daily deals section	✅ Done	Full deals page with filtering, sorting, expiry tracking, stats cards
Coupon integration	❌ Not Done	No coupon/promo code system — no DB table, no UI, no validation logic

Week 8 — Monetization & Analytics (60%)

TASK	STATUS	NOTES
Premium store listings	🟡 Partial	<code>is_premium</code> and <code>is_featured</code> DB fields exist; filtering in store listing; no upgrade/payment flow
Loyalty program / Cashback	❌ Not Done	Zero implementation
Store dashboard (merchant analytics)	✅ Done	Revenue charts, product performance, click/conversion rates, transaction history
Admin analytics dashboard (traffic, search logs, user activity)	✅ Done	KPI cards, revenue charts, user growth, category distribution, search analytics, recent activity
SEO optimization	🟡 Partial	Root metadata only. Missing : dynamic per-page metadata, sitemap, robots.txt, JSON-LD structured data

Week 9 — Advanced Comparison (70%)

TASK	STATUS	NOTES
Comparison tables (specs & prices)	✅ Done	Up to 4 products side-by-side; dynamic spec rows with 40+ translated keys (<code>src/app/[locale]/(dashboard)/compare/page.tsx</code>)
Comparison across stores (warranty, shipping, return policies)	🟡 Partial	DB fields exist (<code>warranty_info</code> , <code>return_policy</code> , <code>delivery_info</code>); <code>ComparisonCard</code> shows warranty/delivery icons. Full detailed policy comparison UI not built.
Delivery time comparison	🟡 Partial	<code>delivery_time_days</code> and <code>delivery_cost</code> fields in DB; limited exposure in end-user UI
Multi-store cart	✅ Done	Fully implemented with store-based grouping, gift wrapping support, localStorage persistence (<code>src/lib/cart/</code>)

TASK	STATUS	NOTES
Gift option integration	✅ Done	Gift wrapping toggle + message + Web Share API in <code>src/components/products/gift-option.tsx</code>

Week 10 — Performance, Compliance & Launch (25%)

TASK	STATUS	NOTES
Mobile app launch (iOS & Android)	❌ Not Done	No React Native / Expo / mobile app codebase
Mobile optimization (responsive, feature parity)	✅ Done	Extensive Tailwind responsive breakpoints, mobile dialogs, responsive grids
Performance optimization (< 3s search results)	🟡 Partial	Turbopack in dev; no evidence of Lighthouse auditing, bundle analysis, or load time benchmarks
Scalability (1M+ users)	🟡 Partial	Supabase-backed (horizontally scalable); no load testing or CDN configuration evidence
Reliability (99.9% uptime)	❌ Not Done	No monitoring service (Sentry, Datadog, etc.) integrated
Cross-browser compatibility	🟡 Partial	Feature detection for modern APIs (BarcodeDetector, SpeechRecognition, Web Share); no polyfills for legacy browsers
Accessibility compliance (WCAG 2.1)	🟡 Partial	Radix UI provides baseline a11y; <code>prefers-reduced-motion</code> supported. Missing comprehensive ARIA landmarks, labels, focus management, and contrast auditing.
Energy efficiency optimization	❌ Not Done	No specific infrastructure optimization evidence
Legal compliance (Saudi regulations)	✅ Done	Privacy policy + Terms of Service (bilingual)
Localization (Hijri/Gregorian + Arabic numerals)	❌ Not Done	Standard Gregorian dates only; no Hijri calendar or Eastern Arabic numeral support
Full Go-Live	❌ Not Done	Not launched

Functional Requirements Checklist

#	REQUIREMENT	STATUS	DETAILS
1	User Registration & Authentication (email, phone, social)	✅	Email, Phone OTP, Google/Facebook/Apple OAuth
2	Guest Access	✅	Public routes accessible without auth
3	Product Search by category	✅	Category filters scraped results end-to-end via <code>searchAllStores()</code> + <code>matchesCategory()</code>
4	Brand & Model Filtering	✅	Dynamic brand checkboxes from DB
5	Advanced Filtering (RAM, storage, resolution, color)	✅	Dynamic spec filters per category with client-side extraction from titles; 6 category configs
6	Price Comparison Engine	✅	Multi-store pricing with best-price highlighting
7	Sort & Filter Results	✅	4 sort options + 7 filter types
8	Store Integration (APIs/web scraping)	✅	All 5 stores: search scrapers + cron scrapers fully implemented
9	Redirection to Store (click-through)	✅	Affiliate URL generation with click tracking
10	Product Detail Page	✅	Full page with images, specs, multi-store prices
11	Wishlist/Save Products	✅	Full CRUD with notes
12	Notification System (price drop, back in stock)	✅	In-app notifications + cron-based alerts
13	Multi-language Support (Arabic/English)	✅	19 namespace files, RTL/LTR
14	Currency Handling (SAR)	✅	<code><Price></code> component with SAR symbol
15	Search Suggestions (auto-suggest)	✅	Debounce autocomplete from DB

#	REQUIREMENT	STATUS	DETAILS
16	User Reviews & Ratings	✓	Product + Store reviews with multi-field ratings
17	Comparison Table (side-by-side)	✓	Up to 4 products, dynamic spec rows
18	Commission Tracking	✓	Full click → conversion tracking with commission calculation
19	Analytics Dashboard (Admin)	✓	KPIs, charts, search analytics, activity logs
20	Store Onboarding Portal	✓	Full store portal with product management
21	Favorites & Personalized Recommendations	●	Basic category-based recs; no AI
22	Mobile Responsiveness	✓	Extensive responsive design
23	Future Mobile App Support	✗	No mobile app codebase
24	Barcode/QR Code Scanner	✓	BarcodeDetector API + manual fallback
25	Search by Voice	✓	SpeechRecognition API (ar-SA / en-US)
26	Price History Tracking	✓	SVG charts with 30/90/365-day ranges
27	Daily Deals Section	✓	Full deals page with expiry tracking
28	Email/SMS Alerts	●	Templates built; no email service connected; SMS = OTP only
29	Comparison Across Stores (warranty, returns, delivery)	●	DB schema ready; UI partially done
30	Favorites Sync (cross-device)	✓	Supabase-backed sync
31	Multiple Login Options (Google, Apple, Facebook, email)	✓	All 4 + phone OTP
32	Affiliate Tracking	✓	Unique click_id per redirect; conversion API
33	Product Availability Checker	✓	4 statuses with colored badges
34	Dynamic Search Filters	✓	Spec filters change dynamically per category; brands + specs + discount/condition/shipping
35	Product Videos/Demos	✓	YouTube embed + direct video player
36	Store Rating System	✓	Multi-field ratings (delivery, quality, service)
37	Smart Recommendations (AI-based)	✗	No AI/ML engine
38	Coupon Integration	✗	Not implemented
39	Multi-Store Cart (Phase 2+)	✓	Store-grouped cart with gift wrapping
40	Comparison by Delivery Time	●	DB fields exist; limited UI
41	Localized Store Integration	✓	5 Saudi stores with full cron + search scrapers
42	Gift Option Integration	✓	Gift wrapping + message + share
43	Search History Log	✓	Auto-logged + saved searches
44	Single Sign-On (OAuth)	✓	Google, Facebook, Apple via Supabase
45	Premium Store Listings	●	DB fields + filtering; no payment flow
46	Loyalty Program / Cashback	✗	Not implemented

Non-Functional Requirements Checklist

#	REQUIREMENT	STATUS	DETAILS
1	Performance (< 3s search results)	●	Turbopack in dev; no benchmarking evidence
2	Scalability (1M+ users)	●	Supabase scales; no load testing
3	Security (SSL, encryption, secure login)	✓	Supabase SSL, RLS, secure auth flows
4	Reliability (99.9% uptime)	✗	No monitoring service (Sentry, Datadog, etc.)
5	Data Accuracy (regular refresh)	✓	All 5 store cron scrapers fully implemented for automated price updates and product discovery

#	REQUIREMENT	STATUS	DETAILS
6	Usability (intuitive bilingual UI)	✓	Clean UI with full AR/EN support
7	SEO Optimization	●	Root metadata only; missing sitemap, robots.txt, JSON-LD
8	Accessibility (WCAG 2.1)	●	Radix UI baseline; lacks comprehensive ARIA
9	Maintainability (modular codebase)	✓	Well-organized: components, lib, app routes
10	Legal Compliance (Saudi e-commerce laws)	✓	Privacy + Terms pages (bilingual)
11	Cross-Browser Compatibility	●	Feature detection; no legacy polyfills
12	Mobile Optimization	✓	Responsive Tailwind design throughout
13	Data Backup (daily automated)	✗	Relies on Supabase managed backups (not custom)
14	Monitoring & Logging	●	Audit logs exist; no external monitoring (Sentry, etc.)
15	Energy Efficiency	✗	No specific optimization evidence
16	Localization (Hijri/Gregorian + Arabic numerals)	✗	Gregorian only; no Hijri or Eastern Arabic numerals

Completion by Week

WEEK	FOCUS AREA	COMPLETION
Week 1	Foundation & Setup	95%
Week 2	User Accounts & Access	100%
Week 3	Search & Filtering	95%
Week 4	Store Integration	95%
Week 5	Product Experience	95%
Week 6	Personalization & Favorites	55%
Week 7	Notifications & Engagement	55%
Week 8	Monetization & Analytics	60%
Week 9	Advanced Comparison	70%
Week 10	Performance, Compliance & Launch	25%

Top Priority Gaps

These are the highest-impact missing features ranked by business value:

- Email service integration** — Templates and infrastructure are built but no email provider (Resend/SendGrid) is connected. This blocks all user-facing email alerts.
- SEO** — No sitemap, robots.txt, dynamic metadata, or JSON-LD structured data. Critical for organic traffic.
- Coupon integration** — Listed as a requirement but completely absent.
- Push notifications** — Settings UI exists; no service worker or Web Push API backend.
- Premium store upgrade flow** — DB fields exist; needs payment gateway integration.
- AI recommendations** — No ML-based recommendation engine; only basic category/view-count logic.
- Monitoring & reliability** — No Sentry, Datadog, or uptime monitoring. Required for production.
- Loyalty program / Cashback** — No implementation at all.
- Mobile app** — iOS/Android app is a PDR requirement but no codebase exists.
- Hijri calendar & Arabic numerals** — Saudi-specific localization requirement not addressed.

Recently Completed (Phase 1 gaps resolved)

- ~~Cron scrapers~~ — All 5 store scrapers (Jarir, Noon, Amazon, Extra, Almanea) fully implemented
- ~~Advanced spec filtering~~ — Dynamic spec filter UI per category with client-side extraction
- ~~Category search filtering~~ — Category passed end-to-end through search pipeline
- ~~Phone password reset~~ — 3-step OTP flow with `reset-password-phone` API route

Legend:  = Implemented |  = Partial |  = Not Implemented

Etlag · Confidential · Tawveeri Project